Chapter 5
Parsers: Top-Down Parsing, FIRST & FOLLOW, LL(1)

From tokens to a parse tree, one idea at a time: what a parser decides, how a top-down parser decides it with one token of lookahead, and how FIRST and FOLLOW make that decision mechanical

Compiler Construction Module

Covers the lecture "Parsers" (the slides are titled Lecture 4). Rebuilt in concept order, not slide order.

How to read this chapter

The lexer (Chapters 2–4) turns characters into tokens. The **parser** takes those tokens and checks that they form a sentence of the language, building a *parse tree* that shows how. This chapter covers top-down parsing, which is what the lecture develops.

The order here is deliberately not the slides' order. The slides announce on slide 6 that a grammar must have "no left recursion and no non-determinism", then define FIRST and FOLLOW twelve slides later, and never say how to repair a grammar. You cannot understand why left recursion is fatal, or how to check that a repair worked, without FIRST, FOLLOW and the LL(1) condition. So this chapter goes:

1. **Grammars** (§1): the notation everything else uses, including the Greek letters and primes that appear everywhere later.
2. **What a parser decides** (§2): top-down versus bottom-up.
3. **Top-down parsing by hand and in code** (§3): the recursive descent parser, whose code asks exactly two questions about the grammar.
4. **FIRST** (§4) and **FOLLOW** (§5): the answers to those two questions, with every slide exercise worked out.
5. **The LL(1) parser** (§6): the table, the condition, the stack algorithm.
6. **When a grammar is not LL(1)** (§7): left recursion, common prefixes, ambiguity, and what can be done about each. This is where the slides' slide 6 finally makes sense.
7. **Deciding LL(1) in exam conditions** (§8), then recipes, classic questions and a map to the *FIRST & FOLLOW Advanced Practice Sheet* (§9).

No section uses a term before it is defined. Every FIRST set, FOLLOW set, table and trace was checked by a program, and the slides' parser was compiled and run.

Slides	Topic	Section here
1–5	parser, parse tree, top-down vs bottom-up, classification	§1, §2
6	requirements: no left recursion, no non-determinism	§7
7–16	recursive descent: code, trace, call stack	§3
17–20	LL(1) parser, FIRST, FOLLOW definitions	§4, §5, §6
21–37	derivation of FIRST and FOLLOW, exercises	§4, §5
38–45	LL(1) table construction, LL(1) parsing	§6
46–66	"is this grammar LL(1)?", homework	§7, §8

1 Grammars, Derivations and Parse Trees

A grammar is a finite set of rules that generates a (usually infinite) set of token strings. Everything in this chapter is a question about a grammar, so the notation has to be solid first.

Concept: Context-free grammar, and the notation used in this chapter

A **context-free grammar** (CFG) has four parts:

- **Terminals**: the tokens the lexer produces, which appear in the final string. Written in lower case or typewriter: a, b, `id`, `+`, `(`.
- **Non-terminals**: names for syntactic categories ("expression", "statement", "the rest of a sum"). Written in upper case: S, A, E, T. A **primed** name such as E' or A' is just another non-terminal; by convention it is a helper introduced when rewriting the grammar, and is read "E-prime". It has no special meaning beyond its own productions.
- **Productions** (rules) $A \to \alpha$: "an A may consist of α". The left side is one non-terminal; the right side α is any string of terminals and non-terminals, possibly empty. $A \to \alpha \mid \beta$ is shorthand for the two productions $A \to \alpha$ and $A \to \beta$; α and β are called the **alternatives** of A.
- A **start symbol** S: the category of a whole program. By convention, the left side of the first production.

Symbols used throughout.

$\alpha, \beta, \gamma, \delta$	Greek letters stand for *any string* of grammar symbols, terminals and non-terminals mixed, possibly empty. "$A \to A\alpha$" means "a production whose right side starts with A and continues with anything".
ε	the empty string. $A \to \varepsilon$ (an **ε-production**) says an A may consist of nothing at all.
\$	the end-of-input marker, treated as a terminal that appears once, after the last token.
$\Rightarrow$	one derivation step (defined below). $\Rightarrow^*$: zero or more steps. $\Rightarrow^+$: one or more steps.

1.1 Derivations

A **derivation step** picks one non-terminal in the current string and replaces it by the right side of one of its productions:

$$\alpha\, A\, \gamma \;\Rightarrow\; \alpha\, \beta\, \gamma \qquad \text{using } A \to \beta.$$

A **derivation** is a chain of steps starting from S. The strings that appear along the way, which still contain non-terminals, are called **sentential forms**. The **language** of the grammar is the set of strings of terminals only that can be derived: $\{\, w : S \Rightarrow^* w \,\}$.

A grammar has no loops; **repetition is expressed by recursion**. The production $E' \to \texttt{+}\, i\, E'$ says "an E' is `+`i followed by another E'", and $E' \to \varepsilon$ ends the chain. Together they mean "zero or more copies of `+`i".

Worked example: The running example: sums

$$E \to i\, E', \qquad E' \to \texttt{+}\, i\, E' \mid \varepsilon.$$

Here i stands for an identifier token. E is "an i followed by the rest of a sum", and E' (the rest) is either nothing or `+`i and more rest. To derive $i + i + i$:

$$E \Rightarrow i\, E' \Rightarrow i + i\, E' \Rightarrow i + i + i\, E' \Rightarrow i + i + i.$$

The last step uses $E' \to \varepsilon$: the non-terminal simply disappears. The language is $\{\, i,\ i+i,\ i+i+i,\ \ldots \,\}$.

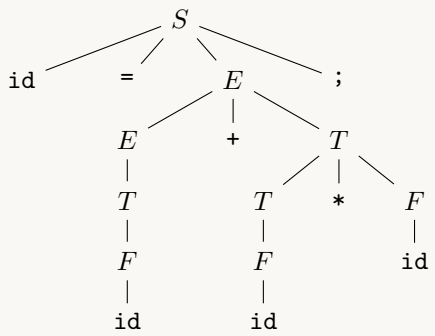
When several non-terminals are present, the order of replacement is a choice. In a **leftmost derivation** the leftmost non-terminal is always replaced first; in a **rightmost derivation**, the rightmost one. Both reach the same strings; they matter because the two families of parsers correspond to the two orders (§2).

1.2 Parse trees

A **parse tree** records a derivation without the order of the steps. The root is S. Each time a non-terminal A is replaced using $A \to X_1 X_2 \cdots X_k$, the node A gets the children $X_1, \ldots, X_k$ in that order (a single child ε for $A \to \varepsilon$). The leaves, read left to right, spell the derived string. Each parse tree corresponds to exactly one leftmost derivation and exactly one rightmost derivation.

Worked example: The slides' first grammar and its parse tree

Grammar: $S \to$ `id = `E `;`, $E \to E$`+`$T \mid T$, $T \to T$`*`$F \mid F$, $F \to$ `id`. The token string `id = id + id * id ;` has the parse tree



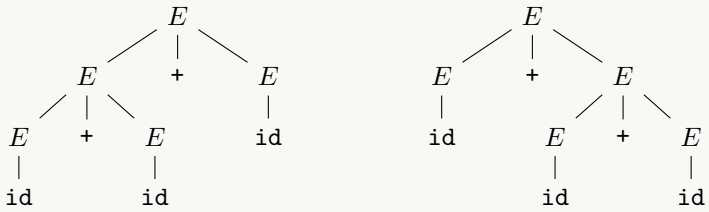
The tree is what the later phases of the compiler need: it shows that `*` groups below `+`, so `id + id * id` means `id` $+$ (`id` $*$ `id`). The grammar encodes operator precedence by putting `*` one level deeper (T inside E).

1.3 Ambiguity

A grammar is **ambiguous** if some string has *two different parse trees* (equivalently, two different leftmost derivations).

Worked example: An ambiguous grammar

$E \to E$`+`$E \mid$ `id` generates the same strings as the sums grammar, but `id + id + id` has two trees:



The left tree reads $(\texttt{id} + \texttt{id}) + \texttt{id}$, the right one $\texttt{id} + (\texttt{id} + \texttt{id})$. For `-` instead of `+` the two trees would compute different values. A parser must produce *one* tree, so an ambiguous grammar leaves it with an unresolvable choice. This is the one grammar defect that no mechanical transformation can repair (§7.3).

2 What a Parser Decides

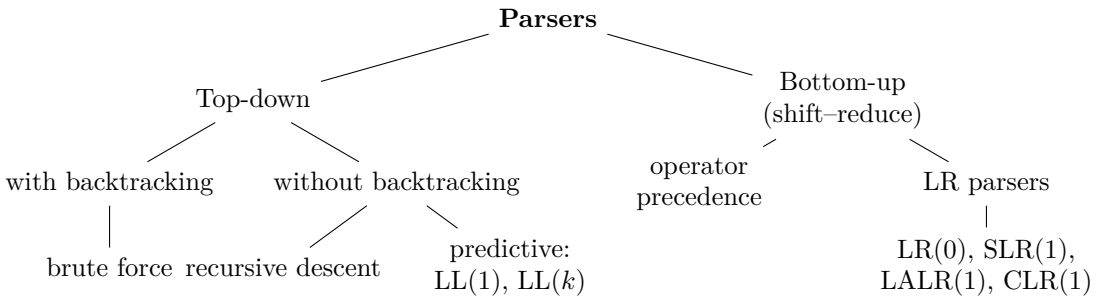
A **parser** is a program that, given a token string, builds a parse tree for it if the string is in the language, and reports an error otherwise. There are two ways to build the tree: from the root down, or from the leaves up.

Worked example: Top-down versus bottom-up on `aabcde`

Grammar: $S \to aABe, \quad A \to Abc \mid a, \quad B \to d.$

Top-down: start at S, expand toward the leaves	**Bottom-up**: start at the leaves, reduce toward S
$S \Rightarrow aABe \Rightarrow aAbcBe \Rightarrow aabcBe \Rightarrow aabcde$	$aabcde \leftarrow aAbcde \leftarrow aAde \leftarrow aABe \leftarrow S$
a **leftmost derivation**, built forwards	a **rightmost derivation**, built in reverse
Decision at each step: **which production** to use for the leftmost non-terminal	Decision at each step: **when to reduce**, and by which production

Read the bottom-up column right to left: $S \Rightarrow aABe \Rightarrow aAde \Rightarrow aAbcde \Rightarrow aabcde$ is a rightmost derivation. A bottom-up parser discovers it backwards, by finding a right side in the current string and replacing it by its left side.



The lecture develops the top-down branch; the names LL(1) and "predictive" are explained in §6. Bottom-up parsers are more powerful (they accept grammars that top-down parsers cannot) and are the subject of a later lecture.

Key insight. A top-down parser's only job is to answer, over and over: *the leftmost non-terminal is A and the next token is a; which production of A do I use?* The whole chapter is about answering that question correctly, in advance, from the grammar alone.

3 Top-Down Parsing, by Hand and in Code

3.1 The guessing game

Parse $i + i$ with the sums grammar $E \to i\,E'$, $E' \to \texttt{+}\,i\,E' \mid \varepsilon$, top-down. Append \$ to the input so that "end of input" is a token like any other: $i + i\,\$$.

- Start at E. It has one production, so there is nothing to decide: $E \Rightarrow iE'$. The first symbol i matches the first token; consume it. Remaining input: $+i\,\$$.
- Now the leftmost non-terminal is E', which has *two* productions. Which one? Look at the next token, `+`. If we chose $E' \to \varepsilon$, E' would vanish and nothing would be left to consume `+`; so the choice must be $E' \to \texttt{+}iE'$. Consume `+` and i. Remaining: \$.
- E' again, next token \$. Now $E' \to \texttt{+}iE'$ cannot work (it needs a `+`), so $E' \to \varepsilon$. The input is exhausted and no non-terminal remains: **accept**.

The token we peeked at before choosing is called the **lookahead**. In this grammar, one token of lookahead always settles the choice: `+` means "more sum", anything else means "the sum is over".

3.2 What happens without lookahead: backtracking

A parser that does not look ahead must *guess* a production and **backtrack** when the guess fails: return to the input position where the guess was made and try the next alternative.

Worked example: Backtracking on `cad`

$S \to c\,A\,d$, $\quad A \to a\,b \mid a$.
1. Expand $S \to cAd$; match `c`.
2. Try $A \to ab$: match `a`; expect `b` but see `d`. **Fail**: go back to just after `c`.
3. Try $A \to a$: match `a`. Then match `d`. End of input: **accept**.

Here one token of lookahead does *not* help: both alternatives of A begin with a. Backtracking is simple but can take exponential time, and undoing work is awkward when the parser also builds trees. Every parser in the rest of this chapter is **predictive**: it chooses correctly the first time from the lookahead, and the grammar is arranged so that this is possible (§7 shows how to fix $A \to ab \mid a$).

3.3 The recursive descent parser

Concept: Recursive descent

A **recursive descent parser** is a top-down parser written as a set of mutually recursive procedures, **one per non-terminal**. The procedure for A looks at the lookahead to choose one of A's productions, then works through that right side from left to right: for a terminal it calls `match`, which checks the token and advances; for a non-terminal it calls that non-terminal's procedure. The program's structure mirrors the grammar's.

Why "recursive descent"? *Descent*: the parse proceeds from the root of the tree down to the leaves. *Recursive*: the procedures call one another, and themselves, exactly as the non-terminals refer to one another. At any moment the program's call stack holds the path from the root of the parse tree to the node being built.

Worked example: The slides' parser for the sums grammar

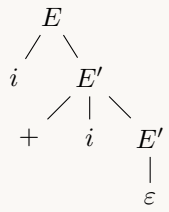
```
E() {                           // E  -> i E'
  if (look_ahead == 'i') {
    match('i');
    E'();
  }
}
E'() {                          // E' -> + i E'
     | eps
  if (look_ahead == '+') {
    match('+');
    match('i');
    E'();
  }
  else
    return;                     // the eps
    choice
}
```

```
match(char c) {
  if (look_ahead == c)
    look_ahead = getchar();     // consume,
    read next
  else
    printf("ERROR!");
}
main() {
  E();
  if (look_ahead == '$')
    printf("Parsing Successful!");
}
```

- `match(c)` consumes one terminal: it checks that the lookahead is `c` and reads the next character.
- `E'()` is the guessing game of §3.1 written as code: $E' \to +iE'$ when the lookahead is `+`, otherwise the ε-production (do nothing, return).
- `main` parses one E and then checks that the whole input was consumed: the next symbol must be `$`.

Worked example: Trace on `i+i$`

Call stack (top on the right)	Lookahead	Action
`main`	`i`	call `E`
`main E`	`i`	`match('i')`, call `E'`
`main E E'`	`+`	lookahead is `+`: `match('+')`, `match('i')`, call `E'`
`main E E' E'`	`$`	lookahead is not `+`: return ($E' \to \varepsilon$)
`main E E'`	`$`	return
`main E`	`$`	return
`main`	`$`	lookahead is `$`: "Parsing Successful!"



The calls visit the parse tree depth first, left to right; each procedure call is one interior node.

Debugging: The slides' code, compiled and run

Compiled with the missing declarations added and nothing else changed, the parser was run on several inputs:

Input	Slides' code	With `look_ahead = getchar()` first	Correct behaviour
`i+i$`	prints nothing	"Parsing Successful!"	accept
`i+$`	prints nothing	"ERROR!Parsing Successful!"	reject
`ii$`	prints nothing	prints nothing	reject
`+i$`	prints nothing	prints nothing	reject

1. **The lookahead is never initialised.** `main` calls `E()` before reading any character, so `look_ahead` is `0`, `E` does nothing, and even valid input produces no output. Fix: `look_ahead = getchar();` at the start of `main`.
2. **An error does not stop the parse.** `match` prints "ERROR!" and carries on, so `i+$` prints "ERROR!Parsing Successful!". Fix: stop (or recover) on error.
3. **Silent failures.** `E` does nothing on a wrong first token, `E'` returns on *any* token other than `+`, and `main` prints nothing on failure. Fix: report an error in each case.

3.4 The two questions the code asks

Here is a corrected version. Read the two tests in `Ep` carefully; they are the heart of the chapter.

```
void E(void) {                          /* E  -> i E'                          */
  if (look_ahead == 'i') { match('i'); Ep(); }
  else error("expected 'i'");
}
void Ep(void) {                         /* E' -> + i E' | eps                  */
  if (look_ahead == '+') { match('+'); match('i'); Ep(); }   /* Q1: can '+' START  + i E' ?   */
  else if (look_ahead == '$') return;                        /* Q2: can '$' come AFTER an E' ? */
  else error("expected '+' or end of input");
}
```

With `main` reading the first character and `error` stopping the program, this accepts `i+i$` and `i+i+i$` and rejects the other inputs with a precise message (checked).

The question we need to answer

To write the procedure for a non-terminal A with alternatives $\alpha_1, \alpha_2, \ldots$, we need, for each alternative and each possible lookahead token a:
Q1 (start). Can a string derived from α_i *begin* with a? If so, $A \to \alpha_i$ is a candidate. The set of such a is called FIRST(α_i) (§4).
Q2 (skip). If α_i can derive the empty string, so that A produces nothing here, can a legally be the token that comes *right after* an A? If so, $A \to \alpha_i$ is a candidate too. The set of such a is called FOLLOW(A) (§5).
For the sums grammar the answers were obvious ({+} and {$}). For a real grammar, with non-terminals that begin with other non-terminals and alternatives that can vanish, the answers must be computed. That is what FIRST and FOLLOW are for. If for every A and a there is *at most one* candidate, the parser never has to guess: the grammar is called **LL(1)** (§6).

3.5 A first obstacle: a procedure that calls itself before reading anything

Try the same recipe on the expression grammar of §1, $E \to E\texttt{+}T \mid T$. The procedure `E()` choosing $E \to E\texttt{+}T$ must, as its first action, call `E()` again, with the *same* lookahead and nothing consumed. That call does the same, forever. The grammar is **left-recursive**: a non-terminal's production begins with that same non-terminal. No lookahead can save a procedure that recurses before it reads.

The fix is to say the same thing without the self-reference at the front. $E \to E\texttt{+}T \mid T$ generates "a T, then zero or more $\texttt{+}T$". We already know how to write "zero or more" top-down: it is exactly the shape of the sums grammar.

$$E \to T\,E', \qquad E' \to \texttt{+}\,T\,E' \mid \varepsilon, \qquad T \to F\,T', \qquad T' \to \texttt{*}\,F\,T' \mid \varepsilon, \qquad F \to \texttt{id} \mid \texttt{(}\,E\,\texttt{)}.$$

This is the grammar the slides use for all their FIRST and FOLLOW work, and now you know where it comes from: it is the §1 expression grammar with the left recursion rewritten as a tail. It generates the same strings (checked on all strings up to 7 tokens). The general recipe, and why the original cannot be made to work, is in §7.1, once FIRST and FOLLOW are available to prove it.

4 FIRST Sets: Which Tokens Can Start It?

4.1 Nullable symbols

Before FIRST, one preliminary notion. A non-terminal A is **nullable** if it can derive the empty string: $A \Rightarrow^* \varepsilon$. A terminal is never nullable. A string α is nullable if every symbol in it is nullable (so ε itself is nullable, and any string containing a terminal is not).

Method: Finding the nullable non-terminals

Mark A nullable if it has a production $A \to \varepsilon$, or a production $A \to Y_1 \cdots Y_k$ whose symbols are all already marked nullable. Repeat until nothing changes.

In the expression grammar, E' and T' are nullable (they have ε-productions); E, T and F are not, because every one of their alternatives contains a non-nullable symbol. Nullability matters because a

nullable symbol at the front of a string can vanish, letting the *next* symbol be the one that starts the string.

4.2 Definition and method

Definition 4.1 (FIRST)**.** For any string of grammar symbols α,

$$\text{FIRST}(\alpha) = \{\, a \text{ terminal} : \alpha \Rightarrow^* a\beta \text{ for some } \beta \,\} \;\cup\; \{\, \varepsilon : \alpha \Rightarrow^* \varepsilon \,\}.$$

In words: the terminals that can appear *first* in something derived from α; plus the marker ε if α can vanish entirely. For a non-terminal A, FIRST(A) is FIRST of the one-symbol string A: the union of FIRST over its alternatives.

The ε in a FIRST set is not a token; it is a flag meaning "this string is nullable". It is kept inside FIRST because it is needed at exactly the moments FIRST is used (the scan below, and Q2 in the table construction).

Method: Computing FIRST

1. If X is a terminal, $\text{FIRST}(X) = \{X\}$.
2. If $X \to \varepsilon$ is a production, add ε to $\text{FIRST}(X)$.
3. For a production $X \to Y_1 Y_2 \cdots Y_k$, scan the right side from the left:
   - add $\text{FIRST}(Y_1) \setminus \{\varepsilon\}$ to $\text{FIRST}(X)$;
   - if Y_1 is nullable, also add $\text{FIRST}(Y_2)\setminus\{\varepsilon\}$; if Y_1 and Y_2 are both nullable, also add $\text{FIRST}(Y_3)\setminus\{\varepsilon\}$; and so on;
   - if *all* of $Y_1, \ldots, Y_k$ are nullable, add ε.
4. Repeat step 3 over all productions until no set changes (a *fixed point*). Sets only grow and are bounded, so this stops.

The same scan gives $\text{FIRST}(\alpha)$ for any string $\alpha = Y_1 \cdots Y_k$, and $\text{FIRST}(\varepsilon) = \{\varepsilon\}$.

Key insight. The scan through $Y_1, Y_2, \ldots$ stops at the first symbol that is *not* nullable, because that symbol will certainly produce something, and whatever comes after it can never be first. The two classic mistakes are stopping too early (ignoring that Y_1 can vanish) and carrying on too far (adding $\text{FIRST}(Y_3)$ when Y_2 cannot vanish).

Worked example: The slides' two warm-ups

- $S \to aABC$, $A \to b$, $B \to c$, $C \to d$: every right side starts with a terminal, so $\text{FIRST}(S) = \{a\}$, $\text{FIRST}(A) = \{b\}$, $\text{FIRST}(B) = \{c\}$, $\text{FIRST}(C) = \{d\}$.
- $S \to ABC$, $A \to a \mid \varepsilon$, $B \to b$, $C \to c$: A is nullable, B is not. Scan ABC: take $\text{FIRST}(A) \setminus \{\varepsilon\} = \{a\}$; A is nullable, so continue to B and take $\{b\}$; B is not nullable, stop. $\text{FIRST}(S) = \{a, b\}$. Neither c nor ε is in it.

Worked example: The expression grammar, in dependency order

$$E \to TE', \quad E' \to \texttt{+}TE' \mid \varepsilon, \quad T \to FT', \quad T' \to \texttt{*}FT' \mid \varepsilon, \quad F \to \texttt{id} \mid (E).$$

Nullable: E', T'. Work from the symbols that depend on nothing upward:

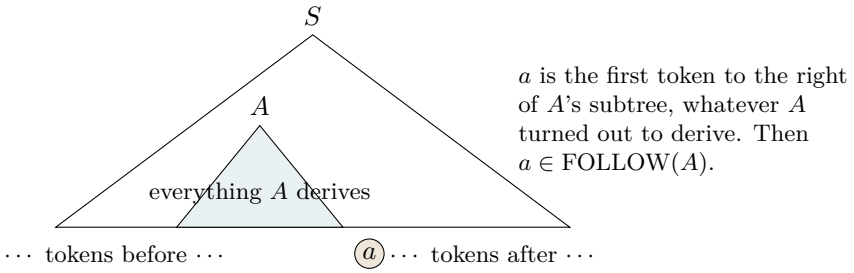
Set	Reason	Result
FIRST(F)	both productions start with a terminal	{`id`, (}
FIRST(T')	$*FT'$ starts with `*`; plus ε for $T' \to \varepsilon$	{`*`, ε}
FIRST(T)	$T \to FT'$: F is not nullable, so stop at F	FIRST(F) = {`id`, (}
FIRST(E')	$+TE'$ starts with `+`; plus ε	{`+`, ε}
FIRST(E)	$E \to TE'$: stop at T	FIRST(T) = {`id`, (}

These are the slides' values. Notice that $\varepsilon \notin \text{FIRST}(T)$ even though T' is nullable: the scan never reaches T'.

5 FOLLOW Sets: Which Tokens Can Come Right After It?

5.1 Why a second set is needed

FIRST answers Q1. For Q2 we need to know, when the parser is about to expand A and the lookahead a cannot start A, whether "A produces nothing and a belongs to what comes next" is a legal reading. That depends not on what A *produces* but on where A is *used*: on what the grammar allows to the right of A.



If A derives ε, its subtree has no leaves, and the parser, standing at A, sees as lookahead the first token to the right of the subtree. FOLLOW collects every token that can stand there.

Definition 5.1 (FOLLOW)**.** For a non-terminal A,

$$\text{FOLLOW}(A) = \{\, a \text{ terminal (including \$)} : S\,\$ \Rightarrow^* \alpha\, A\, a\, \beta \text{ for some } \alpha, \beta \,\},$$

the terminals that can appear *immediately to the right of* A in some sentential form, with \$ included when A can be the last thing before the end of input. FOLLOW sets never contain ε: they are sets of tokens that can actually be seen as lookahead.

5.2 Method

Where can a token to the right of A come from? Look at every place A occurs on a right side, $B \to \alpha\, A\, \beta$. Either the token comes from β, the part of the same production after A; or β vanishes (or is empty, when A is last), in which case the token to the right of A is the token to the right of the whole B, which is by definition in FOLLOW(B).

Method: Computing FOLLOW

1. Put \$ in FOLLOW(S) for the start symbol S.
2. For every occurrence of a non-terminal A on a right side, $B \to \alpha\, A\, \beta$: add FIRST(β) $\setminus \{\varepsilon\}$ to FOLLOW(A).
3. If moreover β is nullable (including $\beta = \varepsilon$, i.e. A is last): add FOLLOW(B) to FOLLOW(A).
4. Repeat over every occurrence of every non-terminal until no set changes.

The slides' three rules are rules 2 and 3 read on examples:

- "the following terminal is selected": rule 2 when β starts with a terminal;
- "the FIRST of the following non-terminal is selected": rule 2 when β starts with a non-terminal. If that FIRST contains ε, keep scanning to the next symbol of β, and if all of β can vanish, apply rule 3;
- "if it is the rightmost in the right side, FOLLOW of the left side is selected": rule 3 with $\beta = \varepsilon$.

Key insight. FIRST flows *up* the grammar (from the symbols on a right side to the left side). FOLLOW flows *down* and *sideways*: whatever can follow B can follow the last non-vanishing symbol of B's right side, because that symbol sits at the right edge of B's subtree. Rule 3 is this downward flow, and it is the rule people forget.

Worked example: The slides' warm-up: $S \to ABC$, $A \to a$, $B \to b \mid \varepsilon$, $C \to c$

FOLLOW(S) = {\$} (rule 1). A is followed by BC: FIRST(BC) = $\{b\} \cup \{c\}$ because B is nullable and the scan continues to C; so FOLLOW(A) = $\{b, c\}$. B is followed by C: FOLLOW(B) = $\{c\}$. C is last in S's production: FOLLOW(C) = FOLLOW(S) = {\$}.

Worked example: The expression grammar, step by step

- FOLLOW(E): \$ by rule 1; `)` because $F \to (E)$ puts `)` right after E. Result {\$, `)`}.
- FOLLOW(E'): E' is last in $E \to TE'$, so it gets FOLLOW(E); it is also last in $E' \to$ `+`TE', which gives it its own set (nothing new). Result {\$, `)`}.
- FOLLOW(T): T is followed by E' in both $E \to TE'$ and $E' \to$ `+`TE'. Rule 2 gives FIRST(E') $\setminus$ $\{\varepsilon\}$ = {`+`}. Since E' is nullable, rule 3 adds FOLLOW(E) and FOLLOW(E'), both {\$, `)`}. Result {`+`, \$, `)`}. (The slide writes FOLLOW(T) = FIRST(E') = {`+`, ε} and circles the ε: that circled ε means "now replace me by FOLLOW of the left side".)
- FOLLOW(T'): last in $T \to FT'$ and in $T' \to$ `*`FT', so it gets FOLLOW(T) = {`+`, \$, `)`}.
- FOLLOW(F): followed by T', so {`*`}; T' is nullable, so add FOLLOW(T) and FOLLOW(T'). Result {`*`, `+`, \$, `)`}.

Production	FIRST of the left side	FOLLOW of the left side
$E \to TE'$	{`id`, `(`}	{\$, `)`}
$E' \to$ `+`$TE' \mid \varepsilon$	{`+`, ε}	{\$, `)`}
$T \to FT'$	{`id`, `(`}	{`+`, \$, `)`}
$T' \to$ `*`$FT' \mid \varepsilon$	{`*`, ε}	{`+`, \$, `)`}
$F \to$ `id` $\mid (E)$	{`id`, `(`}	{`*`, `+`, \$, `)`}

Read the FOLLOW column as answers to Q2: after a complete T the parser may see `+` (another term), `)` (closing a parenthesis) or \$ (end), and nothing else.

5.3 Cycles, and why iteration is needed

Rule 3 creates inclusions $\text{FOLLOW}(B) \subseteq \text{FOLLOW}(A)$, and these can form cycles. Then no order of hand computation finishes in one sweep; the algorithm simply repeats until nothing changes.

Worked example: A FOLLOW cycle

$S \to A\,x \mid B\,y, \quad A \to a\,B, \quad B \to b\,A \mid \varepsilon.$

- Rule 2 gives $x \in \text{FOLLOW}(A)$ (from $S \to Ax$) and $y \in \text{FOLLOW}(B)$ (from $S \to By$).
- Rule 3 gives two inclusions: $A \to aB$ puts B last, so $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$; $B \to bA$ puts A last, so $\text{FOLLOW}(B) \subseteq \text{FOLLOW}(A)$.

Draw a node per non-terminal and an edge $A \to B$ for each inclusion $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$: here $A \to B \to A$ is a **cycle**. Every set on a cycle includes every other, so they are *equal*. Hence $\text{FOLLOW}(A) = \text{FOLLOW}(B) = \{x, y\}$, and $\text{FOLLOW}(S) = \{\$\}$.
Iterating (productions top to bottom, updating in place): pass 1 gives $\text{FOLLOW}(A) = \{x\}$, then $\text{FOLLOW}(B) = \{y\}$, then $\text{FOLLOW}(B) = \{x, y\}$ from $A \to aB$, then $\text{FOLLOW}(A) = \{x, y\}$ from $B \to bA$. Pass 2 changes nothing, so the algorithm stops. A different order of productions may need more passes; the final sets are always the same.

Key insight. The fixed point is unique: the algorithm computes the *smallest* sets that satisfy all the rules, and that does not depend on the order. The order only changes how many passes it takes.

5.4 The slides' exercises, solved

Worked example: GATE 2017: $P \to xQRS$, $Q \to yz \mid z$, $R \to w \mid \varepsilon$, $S \to y$. What is $\text{FOLLOW}(Q)$?

Q is followed by RS. $\text{FIRST}(R) = \{w, \varepsilon\}$ gives w; R is nullable, so continue to S: $\text{FIRST}(S) = \{y\}$; S is not nullable, so stop. $\text{FOLLOW}(Q) = \{w, y\}$: **answer (C)**. Option (D) $\{w, \varepsilon\}$ is wrong twice: ε is never in a FOLLOW set, and it misses y. Option (A) $\{R\}$ is not even a set of terminals.

Worked example: $S \to aBDh$, $B \to cC$, $C \to bC \mid \varepsilon$, $D \to EF$, $E \to g \mid \varepsilon$, $F \to f \mid \varepsilon$

Nullable: C, E, F, and D (both E and F are). So $\text{FIRST}(D) = \{g, f, \varepsilon\}$.

- $\text{FOLLOW}(B)$: B is followed by Dh; $\text{FIRST}(Dh) = \{g, f\} \cup \{h\}$ since D is nullable. So $\{g, f, h\}$.
- $\text{FOLLOW}(C)$: C is last in $B \to cC$ (and in $C \to bC$), so $\text{FOLLOW}(C) = \text{FOLLOW}(B) = \{g, f, h\}$.
- $\text{FOLLOW}(D) = \{h\}$. $\text{FOLLOW}(E)$: E is followed by F, giving f, and F is nullable, so add $\text{FOLLOW}(D)$: $\{f, h\}$. $\text{FOLLOW}(F) = \text{FOLLOW}(D) = \{h\}$.

Grammar	FIRST	FOLLOW
$S \to ABCDE$, $A \to a \mid \varepsilon$, $B \to b \mid \varepsilon$, $C \to c$, $D \to d \mid \varepsilon$, $E \to e \mid \varepsilon$	$S\{a,b,c\}$, $A\{a,\varepsilon\}$, $B\{b,\varepsilon\}$, $C\{c\}$, $D\{d,\varepsilon\}$, $E\{e,\varepsilon\}$	$S\{\$\}$, $A\{b,c\}$, $B\{c\}$, $C\{d,e,\$\}$, $D\{e,\$\}$, $E\{\$\}$
$S \to Bb \mid Cd$, $B \to aB \mid \varepsilon$, $C \to cC \mid \varepsilon$	$S\{a,b,c,d\}$, $B\{a,\varepsilon\}$, $C\{c,\varepsilon\}$	$S\{\$\}$, $B\{b\}$, $C\{d\}$
$S \to ACB \mid CbB \mid Ba$, $A \to da \mid BC$, $B \to g \mid \varepsilon$, $C \to h \mid \varepsilon$	$S\{d,g,h,b,a,\varepsilon\}$, $A\{d,g,h,\varepsilon\}$, $B\{g,\varepsilon\}$, $C\{h,\varepsilon\}$	$S\{\$\}$, $A\{h,g,\$\}$, $B\{\$,a,h,g\}$, $C\{g,\$,b,h\}$

All three agree with the slides and were checked by program. Two details to notice: in the first grammar C is not nullable, so FIRST(S) stops at c and does not contain ε. In the third, S *is* nullable (through $S \to ACB$ with all three nullable), so $\varepsilon \in \text{FIRST}(S)$.

6 The LL(1) Parser

6.1 The name

LL(1) reads as three choices:

- the first **L**: the input is scanned from **L**eft to right;
- the second **L**: the parser builds a **L**eftmost derivation, i.e. it is top-down;
- the **1**: it looks at **one** token of lookahead to choose each production, and never backtracks.

LL(k) uses k tokens of lookahead. "Predictive parser" is another name for the same thing: the production is predicted from the lookahead, not guessed. A *grammar* is called LL(1) if an LL(1) parser exists for it, i.e. if one lookahead token always leaves at most one candidate production. Recursive descent (§3) is an LL(1) parser written as procedures; this section writes the same decisions down as a table and drives them with an explicit stack.

6.2 The parsing table

Method: Building the table M

Rows: non-terminals. Columns: terminals and \$. For each production $A \to \alpha$:

1. **(Q1)** for each terminal $a \in \text{FIRST}(\alpha)$, put $A \to \alpha$ in $M[A, a]$;
2. **(Q2)** if α is nullable ($\varepsilon \in \text{FIRST}(\alpha)$), put $A \to \alpha$ in $M[A, b]$ for each $b \in \text{FOLLOW}(A)$, including \$ if it is there.

Empty cells mean "syntax error". If some cell receives two productions, the grammar is **not LL(1)**.

The slides phrase this as "ε-productions are placed under FOLLOW, the remaining productions under FIRST". Rule 2 is wider than that: it applies to every production whose right side *can derive* ε, not only to the literal $A \to \varepsilon$. The slides' own table for $S \to AB$ with A, B nullable places $S \to AB$ under \$, which only rule 2 explains.

Key insight. Why the rules are right. If the lookahead is a and $a \in \text{FIRST}(\alpha)$, then α can start with a, so $A \to \alpha$ is a candidate. If α can vanish and a can follow A, then "A produces nothing here and a belongs to whatever comes next" is a candidate. No other production can lead to a successful parse with a next. LL(1) means there is never more than one candidate.

Worked example: The table for the expression grammar

	`id`	`(`	`)`	`*`	`+`	\$
E	$E \to TE'$	$E \to TE'$				
E'			$E' \to \varepsilon$		$E' \to$ `+`TE'	$E' \to \varepsilon$
T	$T \to FT'$	$T \to FT'$				
T'			$T' \to \varepsilon$	$T' \to$ `*`FT'	$T' \to \varepsilon$	$T' \to \varepsilon$
F	$F \to$ `id`	$F \to$ `(`E`)`				

$E \to TE'$ goes under $\text{FIRST}(TE') = \{\texttt{id}, (\}$. $E' \to \varepsilon$ goes under $\text{FOLLOW}(E') = \{), \$\}$, and $T' \to \varepsilon$ under $\text{FOLLOW}(T') = \{+,), \$\}$. No cell holds two productions: the grammar is LL(1).

6.3 The LL(1) condition

Building the whole table is one way to decide whether a grammar is LL(1). The following theorem says the same thing without the table, one non-terminal at a time.

Theorem 6.1 (The LL(1) condition)**.** *A grammar (with every non-terminal reachable and able to derive some terminal string) is LL(1) if and only if, for every non-terminal A and every pair of distinct alternatives $A \to \alpha \mid \beta$:*

1. ***no FIRST/FIRST clash:*** FIRST(α) *and* FIRST(β) *share no terminal;*
2. *at most one of α, β is nullable;*
3. ***no FIRST/FOLLOW clash:*** *if β is nullable, then* FIRST(α) *and* FOLLOW(A) *share no terminal.*

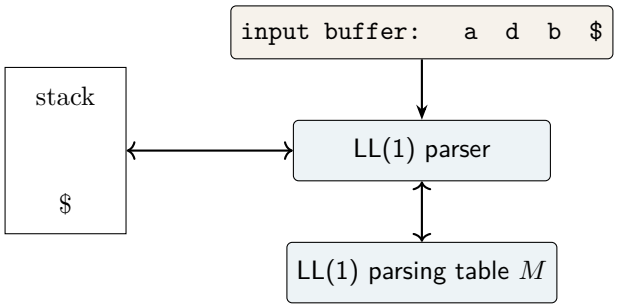
Proof. By the construction, $A \to \alpha$ lands in $M[A, a]$ exactly when $a \in \text{FIRST}(\alpha)$, or α is nullable and $a \in \text{FOLLOW}(A)$. Two alternatives α, β of A share a cell $M[A, a]$ in one of three ways:

- a is in both FIRST sets, which violates condition 1;
- both are nullable and $a \in \text{FOLLOW}(A)$, which violates condition 2 (such an a exists, since a reachable A has a nonempty FOLLOW set);
- one is nullable, say β, $a \in \text{FOLLOW}(A)$, and $a \in \text{FIRST}(\alpha)$, which violates condition 3.

So the table has no doubly-filled cell exactly when the three conditions hold. □

Non-terminals with a single production can never cause a conflict; only those with two or more alternatives need checking.

6.4 The stack algorithm



Instead of the program's call stack (recursive descent), the parser keeps an explicit **stack** holding the part of the sentential form still to be matched, with $ at the bottom. The leftmost unmatched symbol is on top.

Method: The LL(1) parsing algorithm

```
1  push $ then S on the stack; let a be the first token;
2  while true do
3  |  X ← top of the stack;
4  |  if X = $ and a = $ then
5  |  |  accept
6  |  else if X is a terminal then
7  |  |  if X = a then pop X; a ← next token (match);
8  |  |  else error;
9  |  else if M[X, a] = X → Y_1 ··· Y_k then
10 |  |  pop X; push Y_k, ..., Y_1 (so Y_1 is on top)
11 |  else error (empty cell);
```

The right side is pushed *in reverse* so that its first symbol is matched first. Popping X and pushing nothing implements $X \to \varepsilon$.

Worked example: The slides' trace: $S \to aABb$, $A \to c \mid \varepsilon$, $B \to d \mid \varepsilon$, input `adb`

FIRST: $S\{a\}$, $A\{c, \varepsilon\}$, $B\{d, \varepsilon\}$. FOLLOW: $S\{\$\}$, $A\{d, b\}$ (A is followed by Bb, and B is nullable), $B\{b\}$.

	a	b	c	d	$\$$
S	$S \to aABb$				
A		$A \to \varepsilon$	$A \to c$	$A \to \varepsilon$	
B		$B \to \varepsilon$		$B \to d$	

Stack (top right)	Input	Action
$\$\ S$	`a d b $`	$S \to aABb$
$\$\ b\,B\,A\,a$	`a d b $`	match `a`
$\$\ b\,B\,A$	`d b $`	$A \to \varepsilon$
$\$\ b\,B$	`d b $`	$B \to d$
$\$\ b\,d$	`d b $`	match `d`
$\$\ b$	`b $`	match `b`
$\$$	`$`	accept

At the third line the parser must decide whether A produces `c` or nothing; the lookahead `d` is in FOLLOW(A), not in FIRST(c), so A vanishes. That is Q2 in action.

Worked example: `id + id * id` with the expression grammar, and an error

Stack	Input	Action	Stack	Input	Action
$\$\ E$	`id+id*id$`	$E \to TE'$	$\$\ E'T$	`id*id$`	$T \to FT'$
$\$\ E'T$	`id+id*id$`	$T \to FT'$	$\$\ E'T'F$	`id*id$`	$F \to$ `id`
$\$\ E'T'F$	`id+id*id$`	$F \to$ `id`	$\$\ E'T'$`id`	`id*id$`	match
$\$\ E'T'$`id`	`id+id*id$`	match	$\$\ E'T'$	`*id$`	$T' \to *FT'$
$\$\ E'T'$	`+id*id$`	$T' \to \varepsilon$	$\$\ E'T'F*$	`*id$`	match
$\$\ E'$	`+id*id$`	$E' \to +TE'$	$\$\ E'T'F$	`id$`	$F \to$ `id`, match
$\$\ E'T+$	`+id*id$`	match	$\$\ E'T'$ then $\$\ E'$	`$`	$T' \to \varepsilon$, $E' \to \varepsilon$
			$\$$	`$`	accept

On the wrong input `id + * id`, the parser reaches stack $\$\ E'T$ with lookahead `*`. The cell $M[T, *]$ is empty: a T can never start with `*`. The error is reported *at the first token that cannot fit*, which is a strength of predictive parsing (checked by program).

6.5 The table is the recursive descent parser, and vice versa

Each row of the table is one procedure. Row E' of the expression table says: on + use $E' \to +TE'$; on) or \$ use $E' \to \varepsilon$; anything else is an error. As code:

```
void Ep(void) {
  if      (look_ahead == '+')                    { match('+'); T(); Ep(); }  /* FIRST(+ T E')  */
  else if (look_ahead == ')' || look_ahead == '$')  { /* E' -> eps */ }        /* FOLLOW(E')     */
  else    error("expected '+', ')' or end of input");
}
```

So "write a recursive descent parser" and "build the LL(1) table" are the same computation. The table form is what parser generators produce; the procedure form is what people write by hand.

7 When a Grammar Is Not LL(1)

A grammar can fail the LL(1) condition for three reasons. Two of them, left recursion and common prefixes, are *shapes* of the grammar that can be rewritten away without changing the language. The third, ambiguity, cannot. The slides' slide 6 ("no left recursion, no non-determinism") lists the first two as requirements; this section explains them and gives the repairs.

7.1 Left recursion

A grammar is **left-recursive** if some non-terminal can derive a string that starts with itself: $A \Rightarrow^+ A\alpha$. It is *directly* left-recursive if a single production has the form $A \to A\alpha$ (the slides' $E \to E{+}T$), and *indirectly* left-recursive if the loop goes through other non-terminals, for example $S \to Aa$, $A \to Sd$, so $S \Rightarrow Aa \Rightarrow Sda$.

Why it is fatal. §3.5 showed the operational reason: the procedure for A calls itself before consuming a token. Here is the same fact in the language of FIRST sets, which also shows that no cleverness about lookahead can help.

Proposition 7.1 (Left recursion always produces a FIRST/FIRST clash)**.** *If $A \to A\alpha \mid \beta$ and* $\mathrm{FIRST}(\beta)$ *contains a terminal a, then $a \in \mathrm{FIRST}(A\alpha)$ too. Both productions claim the cell $M[A, a]$, so the grammar is not LL(1).*

Proof. $\beta \Rightarrow^* a\gamma$ for some γ, so $A\alpha \Rightarrow \beta\alpha \Rightarrow^* a\gamma\alpha$. Hence $a \in \mathrm{FIRST}(A\alpha)$. □

In fact $\mathrm{FIRST}(A\alpha) \supseteq \mathrm{FIRST}(A) \supseteq \mathrm{FIRST}(\beta)$: the recursive alternative can start with *everything* the non-recursive one can start with, so the lookahead never separates them.

Method: Removing direct left recursion

Group the productions of A into those that start with A (the recursive ones, with tails α_i) and those that do not (the base cases β_j):

$$A \to A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n \qquad (\text{no } \beta_j \text{ starts with } A).$$

Introduce a new helper non-terminal A', meaning "the rest of an A after its base case", and replace them by

$$A \to \beta_1 A' \mid \cdots \mid \beta_n A', \qquad A' \to \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon.$$

Why it is correct. Both versions generate exactly the strings $\beta_j\,\alpha_{i_1}\alpha_{i_2}\cdots\alpha_{i_k}$ for $k \geq 0$: one base case followed by any sequence of tails. The left-recursive version builds this sequence from the right ($A \Rightarrow A\alpha \Rightarrow A\alpha\alpha \Rightarrow \cdots \Rightarrow \beta\alpha\cdots\alpha$); the new version builds it from the left ($A \Rightarrow \beta A' \Rightarrow \beta\alpha A' \Rightarrow \cdots$), the same way the sums grammar does. The language is unchanged; only the shape of the parse tree changes.

Worked example: The expression grammar, again

$E \to E\texttt{+}T \mid T$ has one tail $\alpha = \texttt{+}T$ and one base case $\beta = T$, so it becomes

$$E \to TE', \qquad E' \to \texttt{+}TE' \mid \varepsilon.$$

Likewise $T \to T\texttt{*}F \mid F$ becomes $T \to FT'$, $T' \to \texttt{*}FT' \mid \varepsilon$. This is the derivation of the grammar used in §4–§6. The left-recursive version has FIRST/FIRST clashes on `id` and `(` for both E and T (Proposition 7.1); the transformed one is LL(1), as its table showed. The two generate the same strings (checked on all strings up to 7 tokens).

Method: Removing indirect left recursion

The idea: fix an order of the non-terminals and make sure that no production of A_i starts with an *earlier* non-terminal, by substituting; then any left recursion of A_i has become direct and can be removed as above. Order the non-terminals $A_1, \ldots, A_n$. For $i = 1, \ldots, n$:

1. for each $j < i$, replace every production $A_i \to A_j\gamma$ by $A_i \to \delta_1\gamma \mid \cdots \mid \delta_k\gamma$, where $A_j \to \delta_1 \mid \cdots \mid \delta_k$ are the *current* productions of A_j ("inline A_j");
2. remove the direct left recursion of A_i with the method above.

After step i, no production of A_i starts with any of $A_1, \ldots, A_i$, so no left-recursive cycle can remain once all n steps are done. The algorithm assumes there is no cycle $A \Rightarrow^+ A$ (a non-terminal deriving exactly itself) and no nullable non-terminals at the front of right sides: if some A_j is nullable, a production $A_i \to A_jA_k\gamma$ can hide left recursion through A_k, which must be exposed first.

Worked example: Indirect left recursion, step by step

$S \to Ab \mid c$, $\quad A \to Sa \mid d$. Here $S \Rightarrow Ab \Rightarrow Sab$: indirect left recursion. Order S, A.

- $i = 1$ (S): no production of S starts with S. Nothing to do.
- $i = 2$ (A): inline S in $A \to Sa$, giving $A \to Aba \mid ca \mid d$. Now A is directly left-recursive; remove it (tail $\alpha = ba$, base cases $\beta_1 = ca$, $\beta_2 = d$):

$$A \to caA' \mid dA', \qquad A' \to baA' \mid \varepsilon.$$

The result $S \to Ab \mid c$, $A \to caA' \mid dA'$, $A' \to baA' \mid \varepsilon$ generates the same strings (c, db, cab, $dbab$, …; checked up to length 7). **But it is still not LL(1)**: $S \to Ab$ and $S \to c$ both start with c (because $A \Rightarrow caA'$), a FIRST/FIRST clash; and $b \in \text{FIRST}(baA')$ while also $b \in \text{FOLLOW}(A')$ (from $S \to Ab$ via rule 3), a FIRST/FOLLOW clash. Removing left recursion is *necessary* for top-down parsing, not *sufficient*. Always re-check with Theorem 6.1.

7.2 Common prefixes and left factoring

The slides' "non-determinism" means two alternatives of a non-terminal *start the same way*, so one lookahead token cannot choose between them: a FIRST/FIRST clash that is not caused by recursion.

In $A \to ab \mid a$ (the backtracking example of §3.2), seeing a does not say which production to use.

Method: Left factoring

If $A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \cdots \mid \alpha\beta_n \mid \gamma$ with a common prefix $\alpha \neq \varepsilon$, replace by

$$A \to \alpha A' \mid \gamma, \qquad A' \to \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

(some β_i may be ε). Repeat until no two alternatives share a prefix. The parser now reads α once and *postpones* the choice until after it, when the next token can decide.

Worked example: Two left factorings

- $S \to c\,A\,d$, $A \to a\,b \mid a$ becomes $A \to a\,A'$, $A' \to b \mid \varepsilon$. Check: $\mathrm{FIRST}(b) = \{b\}$ and $\mathrm{FOLLOW}(A') = \mathrm{FOLLOW}(A) = \{d\}$ are disjoint, so A' passes Theorem 6.1: LL(1). The backtracking of §3.2 is gone: after a, the lookahead b or d decides.
- The **dangling else**: $S \to i\,E\,t\,S \mid i\,E\,t\,S\,e\,S \mid a$, $E \to b$ ($i =$ `if`, $t =$ `then`, $e =$ `else`) has the common prefix $iEtS$ and becomes

  $$S \to i\,E\,t\,S\,S' \mid a, \qquad S' \to e\,S \mid \varepsilon, \qquad E \to b,$$

  which is exactly the slides' homework grammar. Left factoring removed the prefix, but §7.3 shows that this grammar still has a conflict, for a deeper reason.

7.3 Ambiguity: the defect no transformation removes

Proposition 7.2. *An ambiguous grammar is never LL(1).*

Proof. Take a string with two different leftmost derivations. They agree up to some step and then expand the same non-terminal A, at the same position, with the same remaining input, by two different productions. Both productions must then be valid choices for that lookahead, so both sit in the same table cell. □

Worked example: Homework: $S \to iEtSS' \mid a$, $S' \to eS \mid \varepsilon$, $E \to b$

FIRST. $\mathrm{FIRST}(S) = \{i, a\}$, $\mathrm{FIRST}(S') = \{e, \varepsilon\}$, $\mathrm{FIRST}(E) = \{b\}$.
FOLLOW.

- $\mathrm{FOLLOW}(S)$ contains \$. In $S \to iEtSS'$, the inner S is followed by S': add $\mathrm{FIRST}(S') \setminus \{\varepsilon\} = \{e\}$, and since S' is nullable, add $\mathrm{FOLLOW}(S)$ (nothing new).
- S' is last in $S \to iEtSS'$, so $\mathrm{FOLLOW}(S) \subseteq \mathrm{FOLLOW}(S')$; S is last in $S' \to eS$, so $\mathrm{FOLLOW}(S') \subseteq \mathrm{FOLLOW}(S)$. A cycle: they are equal, $\{e, \$\}$.
- E is followed by t: $\mathrm{FOLLOW}(E) = \{t\}$.

Table. $S' \to eS$ goes under e; $S' \to \varepsilon$ goes under $\mathrm{FOLLOW}(S') = \{e, \$\}$. The cell $M[S', e]$ holds both: **not LL(1)**, a FIRST/FOLLOW conflict.

	a	b	e	i	t	\$
S	$S \to a$			$S \to iEtSS'$		
S'			$S' \to eS$, $S' \to \varepsilon$			$S' \to \varepsilon$
E		$E \to b$				

Meaning. In `if b then if b then a else a`, the `else` can belong to either `if`: two parse trees, so the grammar is ambiguous, and by the proposition no rewriting of it alone will make it

LL(1). Real parsers resolve the conflict by always choosing $S' \to eS$ on e: the `else` attaches to the nearest unmatched `if`, which is the rule of C, Java and Pascal. With that choice the table is deterministic.

Warning / Gotcha: What the transformations do and do not do

- They preserve the *language*, not the *parse trees*: $E \to TE'$ builds a right-leaning tree for `a+b+c`. Later phases must still treat `+` as left-associative.
- They cannot remove ambiguity. An ambiguous grammar is never LL(1).
- Removing left recursion and left factoring do not guarantee LL(1) (the indirect example above). Always check with FIRST, FOLLOW and Theorem 6.1.
- The slides' very first grammar, $S \to aABe$, $A \to Abc \mid a$, is left-recursive. It illustrates what a top-down tree looks like, but a top-down parser could not use it as written.

8 Is This Grammar LL(1)?

Method: Deciding LL(1) quickly

1. Take the grammar as given; do not transform it unless asked. Compute nullable, FIRST and FOLLOW.
2. For every non-terminal with two or more alternatives, check the three conditions of Theorem 6.1: FIRST sets of the alternatives disjoint; at most one nullable alternative; for the nullable one, the *other* alternatives' FIRST sets disjoint from FOLLOW(A).
3. Non-terminals with a single production can never cause a conflict.
4. If a check fails, name the non-terminal, the two productions and the terminal: that is the conflicting cell.

Grammar	FIRST / FOLLOW	Verdict and reason
$S \to aSbS \mid bSaS \mid \varepsilon$	S: $\{a, b, \varepsilon\}$ / $\{a, b, \$\}$	**Not LL(1)**: $S \to \varepsilon$ goes under FOLLOW $\ni a, b$, colliding with $S \to aSbS$ on a and $S \to bSaS$ on b (FIRST/FOLLOW).
$S \to (S) \mid \varepsilon$	S: $\{(, \varepsilon\}$ / $\{), \$\}$	**LL(1)**: ($\notin$ FOLLOW(S).
$S \to AaAb \mid BbBa$, $A \to \varepsilon$, $B \to \varepsilon$	S: $\{a, b\}$; A, B: $\{\varepsilon\}$ / $\{a, b\}$	**LL(1)**: the two alternatives of S start with a and b (after the nullable A, B vanish).
$S \to A \mid a$, $A \to a$	S, A: $\{a\}$ / $\{\$\}$	**Not LL(1)**: FIRST/FIRST on a (the grammar is also ambiguous).
$S \to aB \mid \varepsilon$, $B \to bC \mid \varepsilon$, $C \to cS \mid \varepsilon$	FOLLOW of all three is $\{\$\}$	**LL(1)**: each ε goes under \$ only.
$S \to AB$, $A \to a \mid \varepsilon$, $B \to b \mid \varepsilon$	A: $\{a, \varepsilon\}$ / $\{b, \$\}$; B: $\{b, \varepsilon\}$ / $\{\$\}$	**LL(1)**: $a \notin$ FOLLOW(A), $b \notin$ FOLLOW(B).
$S \to aSA \mid \varepsilon$, $A \to c \mid \varepsilon$	A: $\{c, \varepsilon\}$ / $\{c, \$\}$	**Not LL(1)**: $c \in$ FOLLOW(A), so $A \to c$ and $A \to \varepsilon$ collide on c.
$S \to A$, $A \to Bb \mid Cd$, $B \to aB \mid \varepsilon$, $C \to cC \mid \varepsilon$	A: $\{a, b, c, d\}$; B / $\{b\}$; C / $\{d\}$	**LL(1)**: FIRST(Bb) $= \{a, b\}$ and FIRST(Cd) $= \{c, d\}$ are disjoint.
$S \to aAa \mid \varepsilon$, $A \to abS \mid \varepsilon$	S: $\{a, \varepsilon\}$ / $\{a, \$\}$; A: $\{a, \varepsilon\}$ / $\{a\}$	**Not LL(1)**: conflicts on a for both S and A.

All verdicts agree with the slides and were checked by building the tables.

Worked example: A bonus from the FIRST/FOLLOW exercises

The slides compute FIRST and FOLLOW for $S \to ACB \mid CbB \mid Ba$, $A \to da \mid BC$, $B \to g \mid \varepsilon$, $C \to h \mid \varepsilon$ without asking whether it is LL(1). It is not: for instance $g \in \text{FOLLOW}(B)$, so $B \to g$ and $B \to \varepsilon$ collide on g, and $S \to ACB$ and $S \to Ba$ both start with g.

9 Tricks, Techniques and Classic Problems

Technique: Exam-speed recipes

- **Nullable first.** List the nullable non-terminals before anything else; every FIRST and FOLLOW computation depends on them.
- **FIRST of $Y_1 Y_2 \cdots$**: take FIRST of each symbol, without ε, *until* you meet one that is not nullable. Add ε only if all of them are nullable.
- **FOLLOW**: go through every *occurrence* of A on a right side. Look right: add FIRST of what follows. If what follows can vanish (or nothing follows), add FOLLOW of the left side.
- **Inclusion graph**: when FOLLOW sets refer to each other, draw edges $A \to B$ for $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$. Sets on a cycle are equal.
- **Table**: each production under FIRST of its right side; if the right side can vanish, also under FOLLOW of its left side.
- **LL(1) check**: only non-terminals with several alternatives matter; name the conflicting cell.
- **Left recursion**: $A \to A\alpha \mid \beta$ becomes $A \to \beta A'$, $A' \to \alpha A' \mid \varepsilon$. Indirect: inline earlier non-terminals, then remove the direct recursion. Re-check LL(1) afterwards.
- **Common prefix**: $A \to \alpha\beta_1 \mid \alpha\beta_2$ becomes $A \to \alpha A'$, $A' \to \beta_1 \mid \beta_2$.

Worked example: Classic short questions

P1. Can a left-recursive grammar be LL(1)? No, by Proposition 7.1 (as long as the non-recursive alternative can start with a terminal).
P2. Can an ambiguous grammar be LL(1)? No (§7.3).
P3. Can ε be in a FOLLOW set? Never. FOLLOW contains tokens that can be seen next, and \$ plays the role of "nothing left".
P4. Is $\text{FOLLOW}(S) = \{\$\}$ always? No. \$ is always in it, but if S appears on a right side it can get more: $S \to (S) \mid \varepsilon$ has $\text{FOLLOW}(S) = \{), \$\}$.
P5. If $\text{FIRST}(A) = \text{FIRST}(B)$, is $\text{FOLLOW}(A) = \text{FOLLOW}(B)$? No. FOLLOW depends on where a non-terminal is *used*, FIRST on what it *produces*. In $S \to Ax \mid By$, $A \to a$, $B \to a$, the FIRST sets are both $\{a\}$ but the FOLLOW sets are $\{x\}$ and $\{y\}$.
P6. Do FIRST/FIRST-disjoint alternatives guarantee LL(1)? No: condition 3 (FIRST/-FOLLOW) can still fail, as in $S \to aSA \mid \varepsilon$, $A \to c \mid \varepsilon$.
P7. Can FIRST and FOLLOW be computed for a left-recursive grammar? Yes. The definitions and the fixed-point algorithms work for any context-free grammar; the grammar just will not be LL(1).
P8. What does an empty table cell mean at run time? A syntax error detected at the current token: no sentence of the language can continue this way.
P9. Does removing left recursion change the language? No. It changes the parse trees (left-leaning becomes right-leaning), which matters for associativity in later phases, not for which strings are accepted.

Warning / Gotcha: Gotchas

- Putting ε in a FOLLOW set, or writing ε as a table column.
- Forgetting \$ in FOLLOW(S), or forgetting to propagate it through rule 3.
- Stopping the FIRST scan at a nullable symbol, or not stopping at a non-nullable one.
- Using only the *first* occurrence of a non-terminal when computing its FOLLOW; every occurrence counts.
- Placing only literal $A \to \varepsilon$ under FOLLOW: every production whose right side is nullable goes there.
- Computing FOLLOW in one sweep when the rules form a cycle; iterate until nothing changes.
- Pushing a right side onto the stack in the wrong order: push it reversed so its first symbol is on top.
- Declaring a grammar LL(1) right after removing left recursion or left factoring, without rechecking.

Map to the FIRST & FOLLOW Advanced Practice Sheet

Exercise	Prepared in
1–2: nullable symbols, long-range propagation, FIRST of strings	nullable symbols and the FIRST method (§4); FOLLOW rule 3 and when \$ propagates (§5)
3: fixed-point iteration, cyclic dependencies	the cycle example and the inclusion graph (§5)
4: eliminating indirect left recursion	the indirect algorithm and its worked example; Proposition 7.1; Theorem 6.1 (§7.1, §6)
5: left factoring a statement grammar	left factoring and the dangling else (§7.2, §7.3)
6: marking a student's table	the gotchas above
7: true or false	classic questions P1–P9
8: table construction, conflict diagnosis	the table method, Theorem 6.1, the LL(1) check (§6, §8)
9: designing a grammar from its sets	run the FOLLOW rules backwards: to put b in FOLLOW(X), write X just before b in some production, or last in a production of a non-terminal whose FOLLOW contains b

Notes on the original slides

- **Order.** Slide 6 requires "no left recursion, no non-determinism" before FIRST, FOLLOW or LL(1) are defined, and no slide says how to remove either. This chapter defers both to §7, after the tools needed to understand and verify them.
- **Recursive descent code.** `main` never reads the first character, so the parser prints nothing even on valid input; `match` prints "ERROR!" but does not stop; failures produce no message. All three were confirmed by compiling and running the code (§3).
- **The first example grammar** $S \to aABe$, $A \to Abc \mid a$ is left-recursive. It illustrates the shape of a top-down tree, but a top-down parser could not use it as written.
- **FOLLOW rules.** The three rules on the slides omit the case where the following non-terminal is nullable; the slides apply it in the FOLLOW(T) example by circling the ε. Rule 3 of §5 states it in general.
- **Table rule.** "ε-productions are placed under FOLLOW" should read "productions whose right

side can derive ε". The slides' own table places $S \to AB$ under \$.

- **Non-determinism** on slide 6 means alternatives with a common prefix. Ambiguity is a separate obstacle that left factoring cannot remove (the homework grammar).

Summary: Chapter 5 in nine lines

1. A grammar generates strings by derivation; a parse tree records a derivation; a grammar is ambiguous if some string has two trees.
2. A parser builds a parse tree from tokens. Top-down builds a leftmost derivation and decides *which production*; bottom-up builds a rightmost derivation in reverse and decides *when to reduce*.
3. A recursive descent parser has one procedure per non-terminal and chooses a production from one lookahead token. Its tests ask two questions: can this token *start* the alternative (FIRST)? can it *follow* the non-terminal, if the alternative vanishes (FOLLOW)?
4. FIRST(α): terminals that can start α, plus ε if α can vanish. Scan symbols until one is not nullable.
5. FOLLOW(A): terminals that can come right after A, with \$ for the start symbol; never ε. Rule 3 copies FOLLOW of the left side when the rest can vanish. Iterate to a fixed point.
6. Table: $A \to \alpha$ under FIRST(α), and under FOLLOW(A) if α is nullable. LL(1) means no cell holds two productions. The stack algorithm expands by the table and matches terminals; an empty cell is a syntax error at the first impossible token.
7. LL(1) $\iff$ no FIRST/FIRST clash, at most one nullable alternative, no FIRST/FOLLOW clash.
8. Left recursion ($A \to A\alpha \mid \beta$) and common prefixes always break LL(1); rewrite them as $A \to \beta A'$, $A' \to \alpha A' \mid \varepsilon$ and $A \to \alpha A'$, $A' \to \beta_1 \mid \beta_2$. Then re-check.
9. Ambiguity cannot be rewritten away; it is resolved by a convention (nearest `if` for the dangling else).